Avance de la sesión 2: C1, C2 y C3

Proyecto Final Integrador, Análisis de Algoritmos (SIST0175). Grupo 3: alineamiento de lecturas genómicas con k-mers como semillas.

Kuilmer Santiago Hernández Cruz · Juan José Berrio Español · Santiago Serrano Cuellar

Contenido del avance

El enunciado pide para la sesión 2: código de C1, C2 y C3 con pruebas, análisis teórico de C1 a C3 y plan de experimentos con hipótesis y generador. Este documento reúne las cuatro partes:

Criterio de la rúbrica	Peso	Dónde está
C1: recurrencia correcta, caso del Teorema Maestro y código que funciona	35 %	Parte 1 (código y pruebas) y Parte 2, sección C1
C2: método contable correcto e implementación	25 %	Parte 1 y Parte 2, sección C2
C3: recurrencia de DP e implementación	25 %	Parte 1 y Parte 2, sección C3
Plan de experimentos con hipótesis explícitas y generador de datos	15 %	Parte 3

Responsables del avance:

Integrante	Componente
Kuilmer Santiago Hernández Cruz	C1 (merge sort), generador de datos y harness de medición
Juan José Berrio Español	C2 (arreglo dinámico) e índice de semillas
Santiago Serrano Cuellar	C3 (Smith-Waterman) y alineador

El plan de experimentos se redactó entre los tres.

Parte 1. Código y pruebas

Repositorio: `git@github.com:KuilmerHC/genoma-alineamiento.git` (privado). Lenguaje: Python 3.11.

Archivo	Contenido
`c1_mergesort/merge_sort.py`	C1: merge sort con contador de comparaciones
`c2_arreglo_dinamico/arreglo_dinamico.py`	C2: arreglo dinámico con duplicación, contadores de costo y banco de créditos del método contable
`c3_smith_waterman/smith_waterman.py`	C3: Smith-Waterman con tabla completa, retroceso y contador de celdas
`comun/kmers.py`	Codificación de k-mers en 2 bits y extracción con ventana deslizante en Θ(n)
`comun/indice.py`, `comun/alineador.py`	Integración de C1 + C2 (índice de semillas con búsqueda binaria) y C3 (alineamiento de la región candidata)
`datos/generar.py`	Generador de genoma y lecturas con semilla fija
`experimentos/bench.py`, `exp_c1…`, `exp_c2…`, `exp_c3…`	Harness de medición y experimentos de C1–C3

Regla del enunciado: el núcleo de los componentes no usa `sorted`, `list.sort` ni `heapq`. `sorted` aparece solo en las pruebas, como oráculo.

Cómo ejecutar (desde la raíz del repositorio):

```
pip install pytest numpy matplotlib
pytest -q tests/test_c1_mergesort.py tests/test_c2_arreglo_dinamico.py \
          tests/test_c3_smith_waterman.py tests/test_comun_kmers.py
tests/test_comun_indice.py
python -m datos.generar 10000 100 42 0.01
python -m experimentos.exp_c1_mergesort
python -m experimentos.exp_c2_arreglo_dinamico
python -m experimentos.exp_c3_smith_waterman
```

Resultado: 97 pruebas, todas pasan.

Archivo de pruebas	Pruebas	Qué verifican
`test_c1_mergesort.py`	6	Igual que `sorted` (pares (kmer, posición) y strings); casos borde; comparaciones entre $\log_2 n!$ y $n \log_2 n$ con entrada aleatoria; mejor caso exacto $(n/2)\cdot\log_2 n < \log_2 n!$
`test_c2_arreglo_dinamico.py`	15	Lectura y escritura; índice fuera de rango; costo acumulado ≤ 3n hasta n = 10^5; copias 1+2+4+8+16; saldo de créditos nunca negativo
`test_c3_smith_waterman.py`	67	Igual a fuerza bruta (mejor alineamiento global entre todas las subcadenas) en 40 casos; ejemplo clásico de la literatura (puntaje 13); alineamiento reconstruido consistente con el puntaje; ubica lecturas con 3 % de sustituciones y 2 % de indels
`test_comun_kmers.py`	5	Ida y vuelta de la codificación; orden entero = orden alfabético; ventana deslizante = codificación directa
`test_comun_indice.py`	4	Búsqueda binaria igual a búsqueda ingenua; claves ordenadas sin repetir; pipeline completo ubica 100/100 lecturas a ≤ 3 bases

Parte 2. Análisis teórico de C1, C2 y C3

0. Contexto: dónde entra cada componente

```
genoma (n bases) ⟶ k-mers (kmer en 2 bits, posición) ⟶ C1 merge sort
⟶ índice de semillas

(claves ordenadas +

C2 posiciones por k-mer)
lectura (m bases) ⟶ sus k-mers ⟶ búsqueda binaria en el índice ⟶
votos por posición inicial
                  ⟶ región candidata (m + 2·margen bases) ⟶ C3 Smith-
Waterman ⟶ alineamiento
```

- **Codificación** (comun/kmers.py): A=0, C=1, G=2, T=3, y el k-mer se lee como un número en base 4. Un k-mer con k ≤ 31 cabe en 64 bits, comparar dos k-mers cuesta O(1) y el orden de los enteros coincide con el orden alfabético. Una ventana deslizante extrae los n−k+1 k-mers en Θ(n).
- **Índice** (comun/indice.py): se construye en Θ(n log n), dominado por C1, y responde cada consulta con búsqueda binaria en Θ(log u), donde u es el número de k-mers distintos.
- **Alineador** (comun/alineador.py): Smith-Waterman alinea cada lectura solo contra la región candidata, en Θ(m·(m+2·margen)) en vez de Θ(m·n) contra el genoma completo. Esa diferencia es la razón de usar semillas.

C1. Merge sort de k-mers

Código: c1_mergesort/merge_sort.py.

PSEUDOCÓDIGO

```
MERGE-SORT(A):                          MEZCLAR(I, D):
  si |A| ≤ 1: devolver A                  R ← [], i ← 0, j ← 0
  mid ← ⌊|A|/2⌋                            mientras i < |I| y j < |D|:
  I ← MERGE-SORT(A[0..mid))                   si I[i] ≤ D[j]:
R.agregar(I[i]); i++
  D ← MERGE-SORT(A[mid..|A|))                  si no:
R.agregar(D[j]); j++
  devolver MEZCLAR(I, D)                    agregar el resto de I y de D a R
                                            devolver R
```

RECURRENCIA LEÍDA DEL CÓDIGO

Línea de `merge_sort`	Costo
`if len(a) <= 1: return a`	Θ(1)
`a[:mid]`, `a[mid:]` (copiar las mitades)	Θ(n)
dos llamadas recursivas sobre ⌊n/2⌋ y ⌈n/2⌉ elementos	T(⌊n/2⌋) + T(⌈n/2⌉)
`mezclar`: cada iteración agrega un elemento a `res`	Θ(n)

```
T(n) = T(⌊n/2⌋) + T(⌈n/2⌉) + θ(n),     T(1) = θ(1)
```

que para el análisis asintótico se escribe T(n) = 2T(n/2) + Θ(n). Los pisos y techos no cambian el orden.

TEOREMA MAESTRO

Con a = 2 (subproblemas), b = 2 (factor de reducción) y f(n) = Θ(n):

- n^(log_b a) = n^($\log_2$ 2) = n.
- f(n) = Θ(n) = Θ(n^(log_b a)): es el **caso 2**. El trabajo es el mismo en cada uno de los $\log_2$ n niveles del árbol de recursión.
- Por tanto **T(n) = Θ(n^(log_b a) · log n) = Θ(n log n)**.

Comprobación con el árbol de recursión: el nivel i tiene 2^i subproblemas de tamaño $n/2^i$ y cuesta $c \cdot n$ en total. Hay $\log_2 n + 1$ niveles, así que $T(n) = c \cdot n \cdot (\log_2 n + 1) = \Theta(n \log n)$.

MEJOR, PEOR Y CASO ESPERADO

El tiempo es **Θ(n log n) en los tres casos**: las copias de `a[:mid]` y de `res` cuestan Θ(n) en cada nivel sin importar los datos. Lo que sí depende de los datos es el número de **comparaciones** C(n). Para n potencia de 2:

Caso	Datos que lo provocan	Comparaciones por mezcla de tamaño s	C(n)
Mejor	arreglo ya ordenado (o en orden inverso): una mitad se agota primero	s/2	$(n/2) \cdot \log_2 n$
Peor	permutación en la que las mitades se intercalan en cada mezcla (p. ej. 0,4,2,6,1,5,3,7)	s − 1	$n \cdot \log_2 n - n + 1$
Esperado	permutación aleatoria	≈ s − 2	$\approx n \cdot \log_2 n - 1{,}25 \cdot n$

En nuestro caso hay muchos k-mers repetidos, pero ordenamos pares (kmer, posición) y la posición es única, así que todas las claves son distintas y el análisis aplica sin cambios.

- **Memoria extra:** Θ(n) por las listas auxiliares, más una pila de recursión de profundidad $\lceil \log_2 n \rceil$.
- **Estabilidad:** el `<=` de `mezclar` conserva el orden relativo de los elementos iguales.

COTA INFERIOR Ω(N LOG N): ÁRBOL DE DECISIÓN

Todo algoritmo que ordena **solo comparando** pares de elementos se representa con un árbol de decisión binario:

- cada nodo interno es una comparación $x \le y$ y tiene dos hijos (sí / no);
- cada hoja es una permutación de salida;

- una ejecución es un camino de la raíz a una hoja, y su longitud es el número de comparaciones.

1. Para ordenar bien cualquier entrada de n claves distintas, cada una de las **n!** permutaciones tiene que aparecer en al menos una hoja.
2. Un árbol binario de altura h tiene a lo sumo 2^h hojas. Entonces $2^h \geq n!$, es decir **$h \geq \log_2(n!)$**.
3. n! ≥ (n/2)^(n/2), porque los n/2 factores mayores valen al menos n/2. Entonces $\log_2(n!) \geq (n/2)\cdot\log_2(n/2) = \Omega(n \log n)$. Por Stirling, $\log_2 n! = n \log_2 n - n \log_2 e + O(\log n) \approx n \log_2 n - 1{,}44 \cdot n$.

Por tanto **todo ordenamiento por comparaciones hace Ω(n log n) comparaciones en el peor caso**. Merge sort hace $n \log_2 n - n + 1$ en el peor caso, así que es **asintóticamente óptimo**: solo se aleja de la cota en ≈ 0,44·n.

La cota también vale para el **caso promedio**: la profundidad media de las hojas de un árbol binario con n! hojas es ≥ $\log_2 n!$. Pero **no vale para cada entrada**: con la entrada ordenada merge sort hace $(n/2)\cdot\log_2 n < \log_2 n!$. El test `test_mejor_caso_queda_bajo_log2_factorial` lo verifica.

C2. Arreglo dinámico de posiciones (duplicación)

Código: c2_arreglo_dinamico/arreglo_dinamico.py. En el índice, cada k-mer distinto tiene un arreglo con las posiciones donde aparece en el genoma.

OPERACIÓN Y COSTO REAL

`agregar(x)`: si n = capacidad, se reserva un arreglo de capacidad 2·c y se **copian** los n elementos; después se **escribe** x. Modelo de costo:

- 1 por escribir el elemento nuevo;
- 1 por cada elemento copiado al duplicar.

El append número i cuesta i si i−1 es potencia de 2 (hay que duplicar) y 1 en otro caso. El peor caso de una sola operación es Θ(n).

MÉTODO CONTABLE

Se cobran **ĉ = 3 créditos por append**:

Créditos	Uso
1	paga la escritura del elemento nuevo
2	quedan depositados “sobre” ese elemento

Invariante: en todo momento, el saldo del banco es ≥ 0 y alcanza para la siguiente duplicación.

Prueba. Supongamos que el arreglo pasa de capacidad c a 2c, lo que ocurre al insertar el elemento c+1, y hay que copiar c elementos.

- La duplicación anterior fue de c/2 a c. Desde entonces se insertaron c/2 elementos y cada uno dejó 2 créditos: **c créditos**, exactamente lo que cuesta copiar los c elementos.
- Con capacidad inicial 1, la primera duplicación (1 → 2) copia 1 elemento, pagado con los 2 créditos del primer append.

Por inducción sobre las duplicaciones, el saldo nunca es negativo. Como el saldo es créditos cobrados menos costo real y es ≥ 0:

```
costo real (n appends) ≤ 3n   ⇒   costo amortizado por append = O(1)
```

Cálculo exacto: costo real = n + (suma de 2^j para todo $2^j < n$) = n + ($2^{\lceil \log_2 n \rceil} - 1$) < 3n. La razón costo/n se acerca a 3 justo después de una duplicación ($n = 2^j + 1$) y a 2 justo antes ($n = 2^j$).

Verificación en el código: la clase lleva el banco en `saldo` y su mínimo en `saldo_minimo`. El test `test_saldo_de_creditos_nunca_negativo` comprueba saldo ≥ 0 hasta $n = 10^5$.

COTAS

Operación	Peor caso	Amortizado	Mejor caso
`agregar`	Θ(n) (duplica)	Θ(1)	Θ(1)
`a[i]`	Θ(1)	Θ(1)	Θ(1)
n appends	Θ(n)		

Memoria: capacidad < 2n, es decir Θ(n).

C3. Smith-Waterman (alineamiento local por programación dinámica)

Código: c3_smith_waterman/smith_waterman.py.

PROBLEMA

Dada una lectura a (largo m) y una región b (largo n), se busca el par de subcadenas a[i'..i) y b[j'..j) cuyo alineamiento tiene **puntaje máximo**. Los puntajes por columna son:

Columna	Puntaje
bases iguales	+2
bases distintas	−1
hueco (inserción o borrado)	−2

Que sea **local** importa porque la región candidata incluye bases de más a los lados (el margen) y la lectura puede traer errores en las puntas. Por eso no se alinean las cadenas completas.

RECURRENCIA

H[i][j] es el mejor puntaje de un alineamiento local que **termina** exactamente en a[i−1] y b[j−1] (o vacío).

```
H[i][0] = H[0][j] = 0

              ┌ 0                                  empezar de cero
H[i][j] = max │ H[i-1][j-1] + s(a_{i-1}, b_{j-1})  alinear las dos bases
              │ H[i-1][j]   − 2                    a_{i-1} contra un hueco
              └ H[i][j-1]   − 2                    b_{j-1} contra un hueco

s(x, y) = +2 si x = y,  −1 si x ≠ y
```

Respuesta: el máximo de H[i][j] sobre todas las celdas (i, j). El alineamiento se reconstruye retrocediendo desde esa celda: en cada paso se elige la celda de la que vino el valor, hasta llegar a un 0.

SUBESTRUCTURA ÓPTIMA

Sea A un alineamiento óptimo que termina en (i, j) con puntaje H[i][j] > 0. Su última columna tiene que ser una de tres:

1. (a_{i-1}, b_{j-1}): lo que queda, A', termina en (i−1, j−1) y vale H[i][j] − s.
2. (a_{i-1}, −): A' termina en (i−1, j).
3. (−, b_{j-1}): A' termina en (i, j−1).

Si A' no fuera óptimo para su subproblema, se podría cambiar por uno mejor y pegarle la misma última columna. El resultado sería un alineamiento que termina en (i, j) con puntaje mayor que A: contradicción (**argumento de cortar y pegar**). El 0 cubre el caso en que conviene no extender nada, porque todo lo anterior suma negativo. Así, H[i][j] solo depende de soluciones óptimas de subproblemas más pequeños.

SUBPROBLEMAS SUPERPUESTOS

Una recursión directa sobre (i, j) llama a (i−1, j−1), (i−1, j) e (i, j−1), y cada una de ellas vuelve a llamar a (i−1, j−1). El número de llamadas cumple R(i, j) ≥ 3·R(i−1, j−1), es decir R ≥ 3^min(m,n): **exponencial**. Pero solo hay (m+1)(n+1) subproblemas distintos. La tabla calcula cada uno una vez.

TABLA Y ORDEN DE LLENADO

- Tabla H de tamaño (m+1)×(n+1).

- Fila 0 y columna 0 en 0 (caso base).
- Se llena fila por fila, de izquierda a derecha. Así, al calcular H[i][j] ya están listas las tres celdas de las que depende: la diagonal, la de arriba y la de la izquierda.

COMPLEJIDAD

	Mejor	Peor	Esperado
Llenar la tabla	Θ(mn)	Θ(mn)	Θ(mn)
Retroceso	O(1) (sin coincidencias, puntaje 0)	Θ(m + n)	
Memoria	Θ(mn)	Θ(mn)	Θ(mn)

- **Tiempo:** el tiempo no depende de los datos. Siempre se calculan las m·n celdas, cada una en O(1), y el contador `celdas` lo comprueba (= m·n).
- **Memoria:** si solo se necesitara el puntaje, bastarían dos filas: Θ(n) de memoria con el mismo tiempo. Guardamos la tabla completa porque necesitamos el alineamiento.
- **En el pipeline:** m = 50 y la región mide m + 2·10 = 70, así que cada lectura cuesta Θ(m^2) ≈ 3500 celdas. Contra el genoma completo serían m·n = 50·10^6 celdas.

CORRECCIÓN VERIFICADA CON PRUEBAS

tests/test_c3_smith_waterman.py comprueba:

- **Fuerza bruta:** en 40 pares aleatorios de largo ≤ 6, el puntaje coincide con un oráculo que prueba el alineamiento global (Needleman-Wunsch) entre **todos** los pares de subcadenas. Eso es literalmente la definición de alineamiento local.
- **Reconstrucción:** el alineamiento devuelto tiene el puntaje reportado y, quitando los huecos, coincide con las subcadenas a[inicio_a:fin_a] y b[inicio_b:fin_b].
- **Ejemplo clásico de la literatura** (TGTTACGG contra GGTTGACTA con +3/−3/−2): puntaje 13, alineamiento GTT-AC / GTTGAC.
- **Lecturas sintéticas** con 3 % de sustituciones y 2 % de indels: SW las ubica a ≤ 3 bases de su posición real.

Parte 3. Plan de experimentos

Para cada componente se fija una **hipótesis explícita**, derivada del análisis teórico (Parte 2), junto con el experimento que la pone a prueba y el **criterio con que se acepta o se rechaza**. Los experimentos de C1 a C3 ya están implementados y ejecutados; sus resultados preliminares están en la sección 3. Los de C4 a C6 se ejecutan en la sesión 3.

1. Protocolo común

Lo implementa el harness experimentos/bench.py.

Regla (enunciado §4.3)	Cómo se cumple
`time.perf_counter()`, ≥ 5 repeticiones, mediana y dispersión	`medir()` repite 5 veces; `resumen()` reporta mediana, Q1, Q3, mínimo y máximo. Las gráficas muestran la mediana con una barra Q1–Q3
Generación de datos fuera del tiempo medido	`medir(funcion, preparar)`: `preparar()` arma los datos antes de arrancar el cronómetro
Log-log y pendiente estimada	`pendiente_loglog()`: ajuste por mínimos cuadrados de log T = p·log n + b
Para n log n, graficar T(n)/(n log n)	Gráficas `*_razon_*`, con el eje Y desde 0 para no exagerar la variación
≥ 5 tamaños en progresión geométrica	n se duplica: entre 6 y 11 tamaños por experimento
Datos con semilla fija	datos/generar.py: `random.Random(semilla)`

Control de ruido:

- El recolector de basura se apaga durante cada medición.
- Los experimentos se corren uno a la vez, nunca en paralelo, para que no compitan por la CPU.

- Cada script guarda su CSV en `experimentos/resultados/` y sus PNG en `experimentos/graficas/`.

Cómo leer la pendiente de n log n. En log-log, n log n **no** es una recta. Su pendiente local es 1 + 1/ln n. Si se ajusta una recta a $c \cdot n \cdot \log_2 n$ sobre el mismo rango que medimos ($n = 10^3 \ldots 5{,}12 \cdot 10^5$), la pendiente esperada es **≈ 1,10**, no 1. Esa es la referencia contra la que se compara C1.

Reproducir (desde la raíz del repo):

```
python -m experimentos.exp_c1_mergesort
python -m experimentos.exp_c2_arreglo_dinamico
python -m experimentos.exp_c3_smith_waterman
```

2. Hipótesis por componente

H1: C1 MERGE SORT

- **Hipótesis:** $T(n) = \Theta(n \log n)$. El número de comparaciones $C(n)$ cumple $\log_2 n! \le C(n) \le n \log_2 n - n + 1$. En entradas aleatorias, $C(n) \approx n \log_2 n - 1{,}25n$.
- **Datos:** n k-mers (k = 15) codificados en 2 bits, extraídos de un genoma de semilla 42. $n = 1000 \cdot 2^i$ para $i = 0 \ldots 9$, es decir de 10^3 a $5{,}12 \cdot 10^5$ (10 tamaños).
- **Mediciones:** tiempo (mediana y Q1–Q3), comparaciones contadas, $T/(n \log_2 n)$, $C/(n \log_2 n)$.
- **Criterio de aceptación:**
  - la pendiente log-log está en [1,05; 1,20];
  - $T/(n \log_2 n)$ varía menos de ±20 % alrededor de su media;
  - las comparaciones quedan dentro de las dos cotas en todos los tamaños.
- **Para la entrega final:**
  - llegar a $n = 10^6$;
  - medir el mejor caso (entrada ordenada, $C = (n/2) \log_2 n$) y el peor (entrada intercalada, $C = n \log_2 n - n + 1$);
  - medir memoria con `tracemalloc`.

H2: C2 ARREGLO DINÁMICO

- **Hipótesis:**
  - el costo real de n appends (escrituras + copias) es < 3n, y el saldo de créditos nunca es negativo;
  - el tiempo total es Θ(n): pendiente log-log ≈ 1 y tiempo por append aproximadamente constante;
  - la razón costo/n oscila entre ≈ 2 (justo antes de duplicar) y ≈ 3 (justo después, n = 2^j + 1).
- **Datos:** n = 1000·2^i para i = 0…10, es decir de 10^3 a 1,024·10^6 (11 tamaños). Además, n = 2^j + 1 para observar el peor caso de la razón.
- **Mediciones:** tiempo, costo acumulado, costo/n, `saldo_minimo`. También el costo acumulado operación por operación para i ≤ 300 (gráfica en “dientes de sierra” contra la recta 3i).
- **Criterio de aceptación:**
  - la pendiente está en [0,95; 1,05];
  - costo/n < 3 en todos los tamaños;
  - `saldo_minimo` = 0 (nunca negativo).

H3: C3 SMITH-WATERMAN

- **Hipótesis:** T(m, n) = Θ(mn), independiente del contenido de las cadenas.
- **Datos:** cadenas de semillas 1 y 2.
  - (a) Lectura fija m = 50 y región n = 500·2^i, de 500 a 32 000 (7 tamaños): es el caso real del pipeline.
  - (b) Cuadrado m = n = 64·2^i, de 64 a 2048 (6 tamaños).
- **Mediciones:** tiempo, T/(mn) en ns por celda, celdas calculadas (deben ser exactamente m·n).
- **Criterio de aceptación:**
  - pendiente (a) en [0,95; 1,05];
  - pendiente (b) en [1,90; 2,10];
  - T/(mn) varía menos de ±20 % en el caso (a).

- **Para la entrega final:**
  - medir memoria: Θ(mn) con la tabla completa frente a Θ(n) con la versión de dos filas;
  - comparar el costo de alinear contra la región candidata con el de alinear contra el genoma completo.

H4: C4 QUICKSORT ALEATORIZADO (SESIÓN 3)

- **Hipótesis:**
  - con claves distintas, E[comparaciones] = 2(n+1)H_n − 4n ≈ 2n ln n ≈ 1,39·n $\log_2$ n;
  - con partición de 3 vías y muchos k-mers repetidos, el número de comparaciones baja;
  - la varianza entre repeticiones es pequeña: el pivote aleatorio evita depender del orden de la entrada.
- **Datos:** los mismos tamaños que H1, con dos variantes:
  - pares (kmer, posición), donde todas las claves son distintas;
  - solo kmers con k pequeño (k = 6, máximo 4096 claves distintas), con muchos duplicados.
  - Además, entrada ya ordenada: el pivote aleatorio mantiene Θ(n log n) donde un quicksort con pivote fijo sería Θ(n^2).
- **Mediciones:** comparaciones/(n ln n) (debe tender a 2), tiempo frente a C1 sobre los mismos datos y en la misma máquina, dispersión entre semillas.
- **Criterio de aceptación:**
  - comparaciones/(2n ln n) en [0,85; 1,05];
  - quicksort hace más comparaciones que merge sort (con n = $5 \cdot 10^5$ la fórmula exacta da ≈ 1,23·n $\log_2$ n, que tiende a 1,39 cuando n → ∞; merge sort hace ≈ 0,93), y se discute por qué aun así puede ser más rápido o más lento en Python.

H5: C5 EXTERNAL MERGE SORT SOBRE `DISCOSIMULADO` (SESIÓN 3)

- **Hipótesis:** las E/S **contadas** por `DiscoSimulado` coinciden con las predichas: E/S = $2 \cdot (N/B) \cdot (1 + \lceil \log_{(M/B - 1)}(N/M) \rceil)$.
  - El 1 corresponde a la fase de crear las corridas iniciales ordenadas de tamaño M.
  - El logaritmo cuenta las pasadas de mezcla (M/B − 1)-vías.
  - Cada pasada lee y escribe los N/B bloques una vez.
- **Datos:** registros (kmer uint64, posición uint32) de 12 bytes, con B = 64 registros por bloque.
  - $N \in \{10^5; 1{,}78 \cdot 10^5; 3{,}16 \cdot 10^5; 5{,}62 \cdot 10^5; 10^6\}$: progresión geométrica de razón $\sqrt[4]{10}$. Con razón 2 solo entrarían 4 tamaños en el rango obligatorio $[10^5, 10^6]$.
  - M = N/20 (máximo permitido) y M = N/100, para ver cómo cambia el número de pasadas.
- **Mediciones:** lecturas y escrituras contadas, E/S predichas, error relativo, número de pasadas, tiempo.
- **Criterio de aceptación:**
  - error relativo < 2 % cuando N/B y N/M son enteros;
  - las diferencias se explican por los $\lceil\ \rceil$ del último bloque o la última corrida;
  - la salida queda ordenada (verificada contra `sorted` como oráculo en el test).

H6: C6 MAPREDUCE CON `MULTIPROCESSING` (SESIÓN 3–4)

- **Hipótesis:**
  - trabajo $W = \Theta(R \cdot m \cdot (m + 2 \cdot \text{margen}))$ para R lecturas, más el reduce;
  - profundidad $D = \Theta(\lceil R/p \rceil \cdot m \cdot (m + 2 \cdot \text{margen})) + \Theta(p)$ por la combinación;
  - el speedup sigue la ley de Amdahl, $S(p) = 1 / (f + (1 - f)/p)$, donde la fracción serial f incluye construir el índice, arrancar los procesos y serializar los datos (pickle) hacia los workers.
- **Datos:** $R \in \{10^3, 2 \cdot 10^3, \ldots, 1{,}6 \cdot 10^4\}$ lecturas sobre un genoma de 10^6 bases y $p \in \{1, 2, 4, 8, 16\}$ (la máquina de pruebas tiene 16 núcleos lógicos).

- **Mediciones:**
  - T_1, T_p y S(p) = T_1/T_p;
  - eficiencia S(p)/p;
  - tiempo de la parte serial medido por separado, para estimar f directamente y también por ajuste de Amdahl.
- **Criterio de aceptación:**
  - el S(p) medido queda dentro de ±15 % del S(p) predicho por Amdahl con el f medido;
  - el resultado paralelo es idéntico al secuencial.

BONUS: ÁRBOL WAVELET (SOLO SI C1–C6 ESTÁN COMPLETOS)

- **Hipótesis:** rank(c, i) cuesta Θ(log σ) = Θ(2) consultas de rank sobre vectores de bits, es decir O(1) respecto de n; memoria ≈ n·$\log_2$ σ bits.

3. Resultados preliminares de C1, C2 y C3

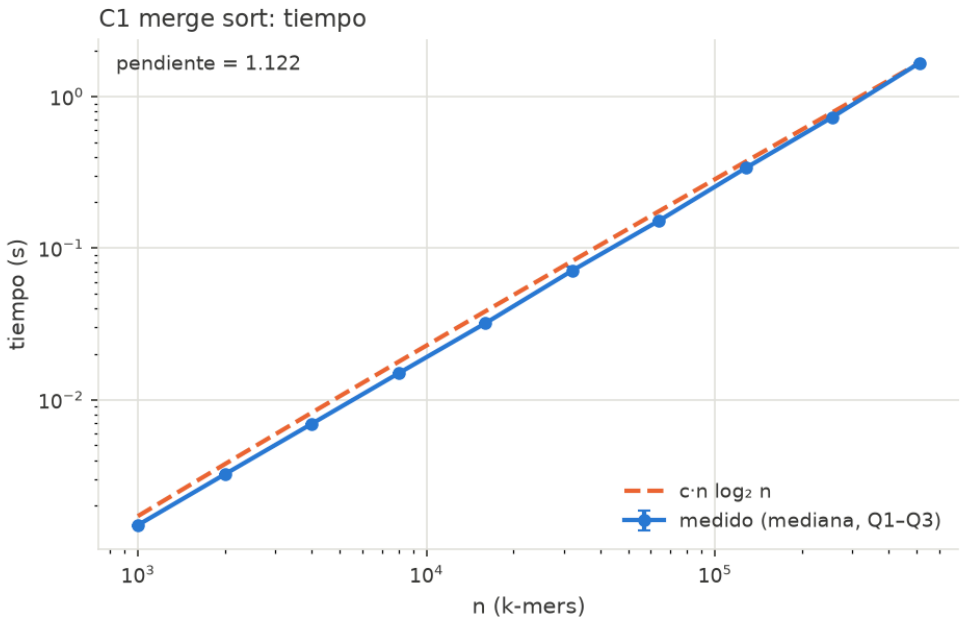
Ejecución en esta máquina, Python 3.11, 5 repeticiones por punto.

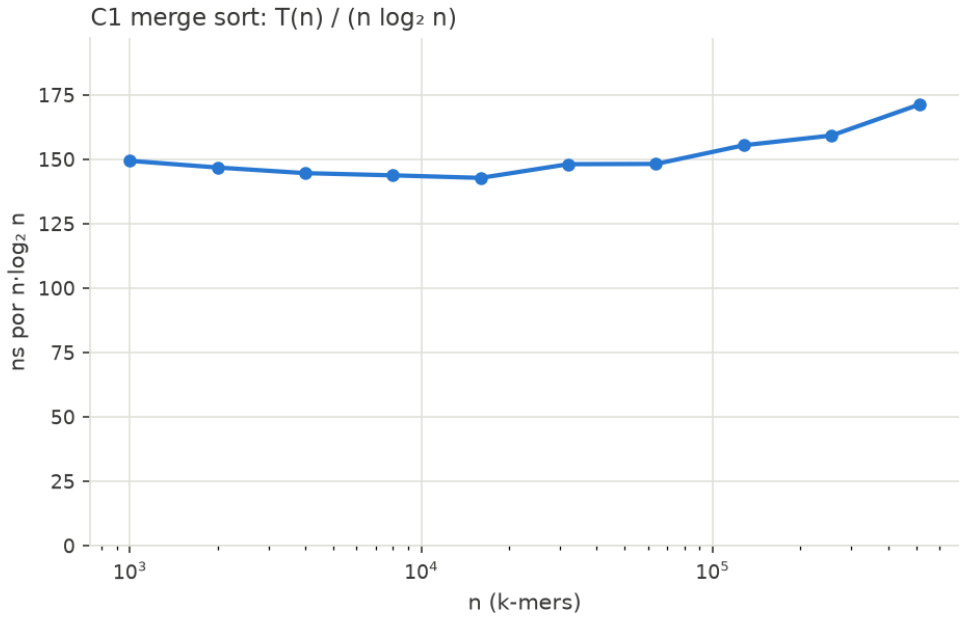
Exp.	Pendiente medida	Esperada	Otras métricas	¿Se acepta?
H1 merge sort	**1,12**	≈ 1,10 (rango [1,05; 1,20])	$T/(n \log_2 n)$ entre 143 y 171 ns (desviación máx. 13 % de la media, dentro de ±20 %); $C/(n \log_2 n)$ de 0,866 a 0,929, tendiendo a $1 - 1{,}25/\log_2 n$ (0,934 en $n = 5{,}12 \cdot 10^5$); $C \geq \log_2 n!$ en todos los n	Sí
H2 arreglo dinámico	**1,01**	1	costo/n = 2,02 en $n = 1000 \cdot 2^i$ y 2,997–3,000 (siempre < 3) en $n = 2^j + 1$; saldo mínimo = 0; 230–250 ns por append (desviación máx. 4 %)	Sí
H3a SW, m = 50	**1,02**	1	T/(mn) entre 133 y 143 ns por celda (desviación máx. 4 %)	Sí
H3b SW, m = n	**2,08**	2	T/(mn) entre 119 y 165 ns por celda (desviación máx. 22 %, ver discrepancias)	Sí

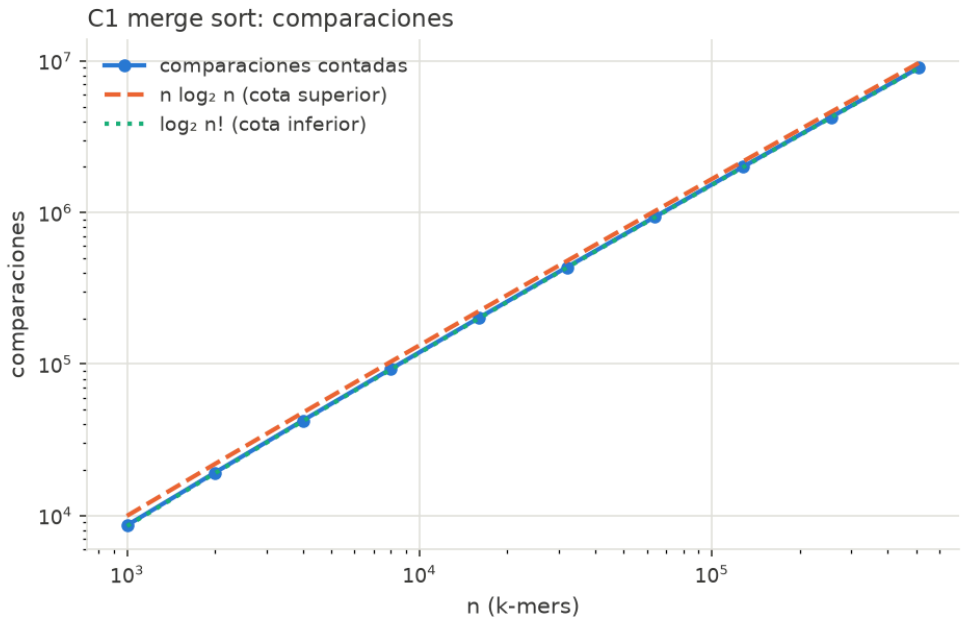
Gráficas: experimentos/graficas/. Datos: experimentos/resultados/.

GRÁFICAS

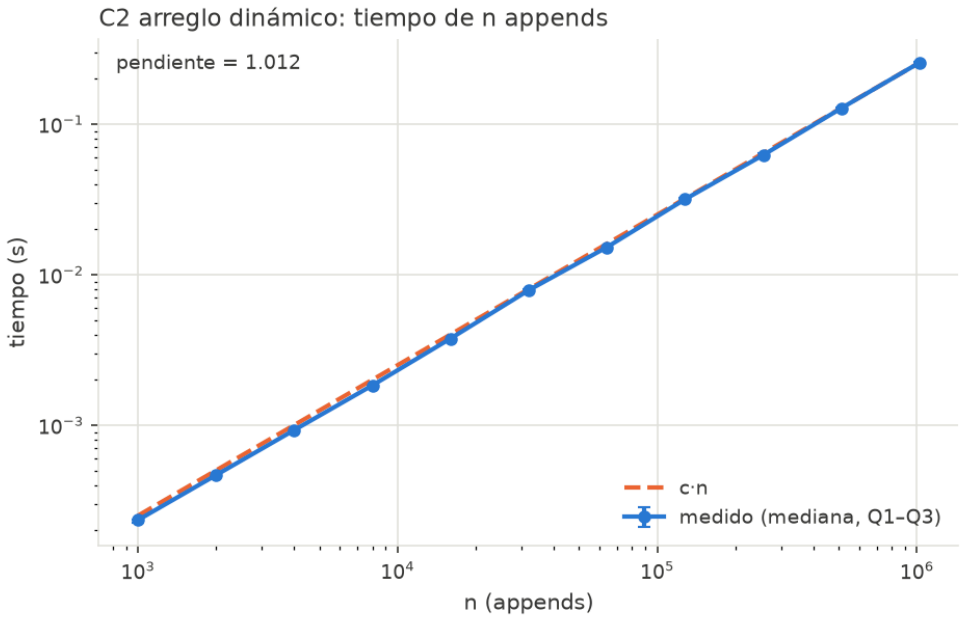
C1, merge sort

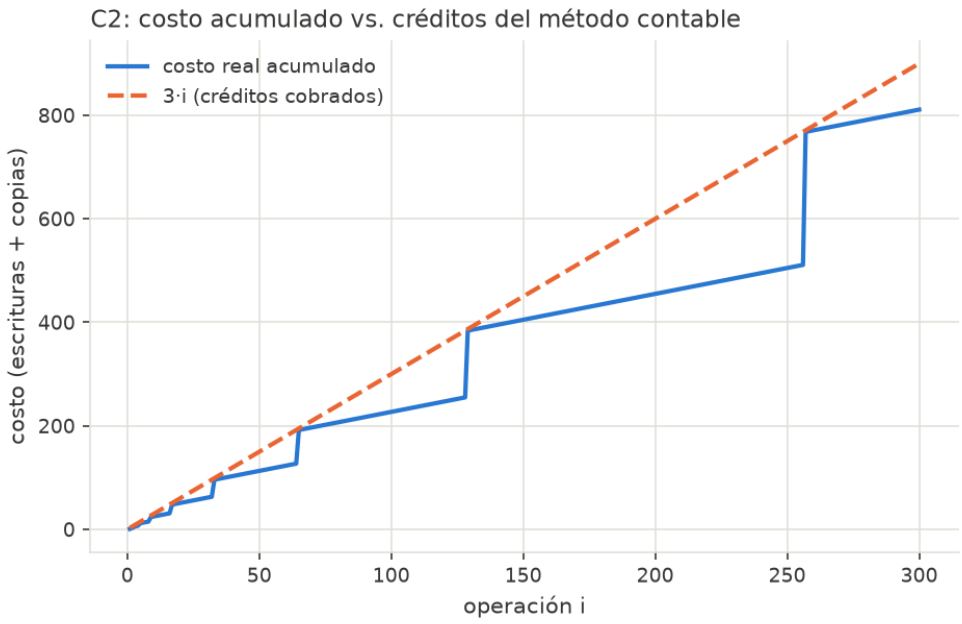

C1 merge sort: tiempo
pendiente = 1.122
tiempo (s)
10^{0}
10^{-1}
10^{-2}
10^{3}
10^{4}
10^{5}
n (k-mers)
c·n log₂ n
medido (mediana, Q1–Q3)


C1 merge sort: T(n) / (n log₂ n)
ns por n·log₂ n
175
150
125
100
75
50
25
0
10^{3}
10^{4}
10^{5}
n (k-mers)



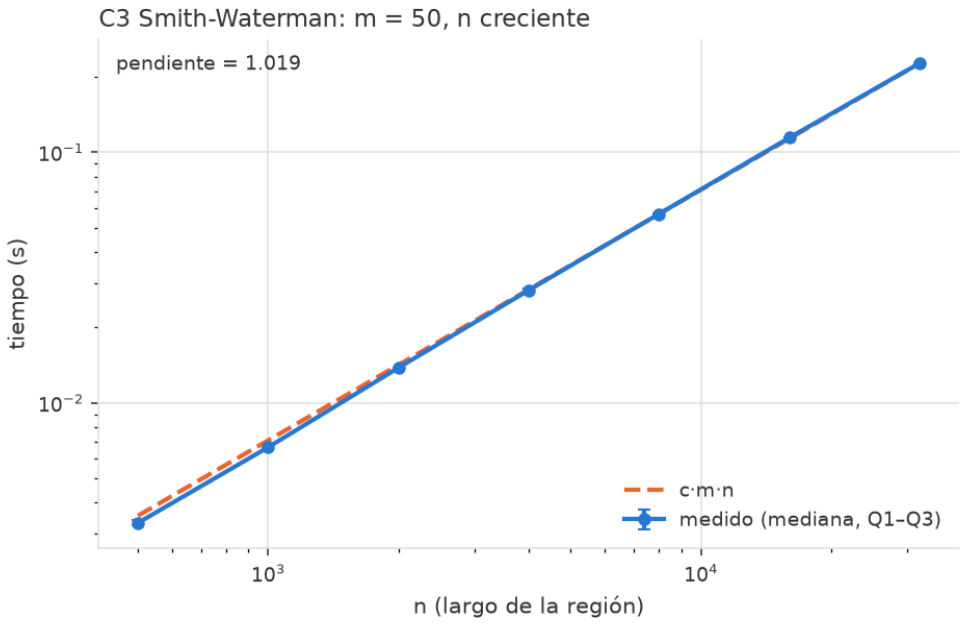
C2, arreglo dinámico

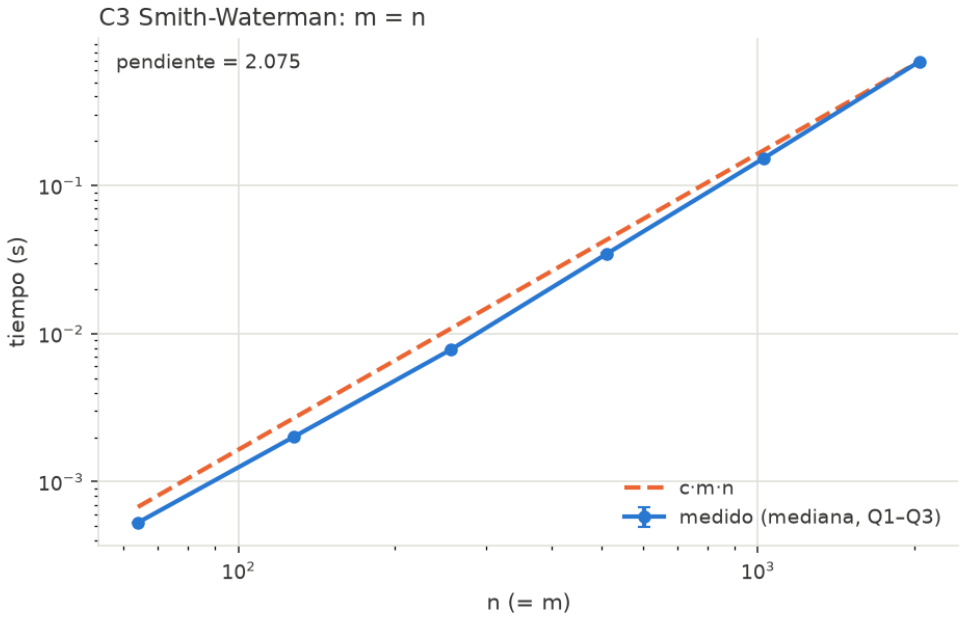


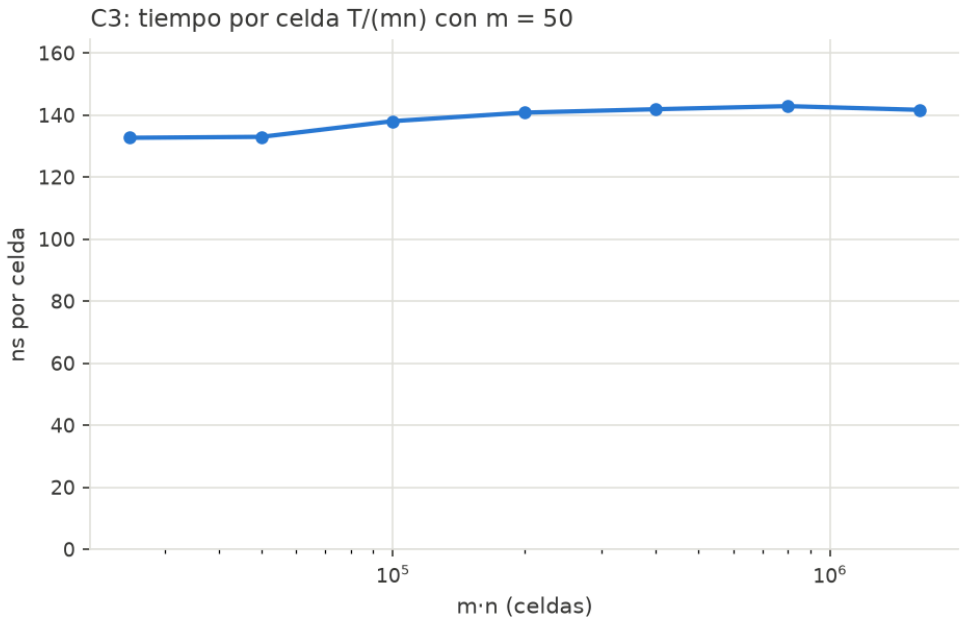




C3, Smith-Waterman


C3 Smith-Waterman: m = 50, n creciente
pendiente = 1.019
10^{-1}
10^{-2}
tiempo (s)
c·m·n
medido (mediana, Q1–Q3)
10^{3}
10^{4}
n (largo de la región)


C3 Smith-Waterman: m = n
pendiente = 2.075
10^{-1}
10^{-2}
10^{-3}
tiempo (s)
c·m·n
medido (mediana, Q1–Q3)
10^{2}
10^{3}
n (= m)



DISCREPANCIAS OBSERVADAS Y EXPLICACIÓN

- **C1, T/(n log n) sube para n ≥ 10^5 (hasta 13 % sobre la media).** Las comparaciones por elemento casi no cambian, así que no es un efecto algorítmico. Es memoria: con n grande, las listas de Python (punteros a tuplas dispersas en el heap) ya no caben en caché, y cada slice y cada lista nueva de `mezclar` reserva y copia más memoria. Para la entrega final lo comprobaremos midiendo con `tracemalloc` y repitiendo con arreglos de NumPy (memoria contigua).
- **C3 cuadrado, el costo por celda sube de 119 a 165 ns.** La tabla H de 2049×2049 enteros de Python ocupa decenas de MB y deja de caber en caché. Con m = 50 la tabla es pequeña y la razón queda casi plana (±4 %). Por eso la pendiente da 2,08 y no 2,00.
- **C2, costo/n ≈ 2,02 y no 3.** No es un error. Con $n = 1000 \cdot 2^i$ el arreglo está justo antes de una duplicación (la capacidad es $1024 \cdot 2^i$), y la cota 3n solo se alcanza con $n = 2^j + 1$. Por eso el experimento mide ambos casos.
- **Constantes del intérprete.** ≈ 140 ns por comparación o por celda es del orden de 100 veces más que en C. Las constantes cambian, pero no las pendientes, que es lo que valida la teoría.

4. Generador de datos

datos/generar.py produce, con semilla fija:

- **Genoma:** n bases uniformes en {A, C, G, T}.
- **Lecturas:** de largo 50, tomadas de posiciones aleatorias del genoma, con tres tipos de error:
  - sustituciones con probabilidad `tasa_error` (2 % por defecto);
  - inserciones y borrados con probabilidad `tasa_indel` (por defecto 0; los experimentos de C3 y el pipeline usan 1–2 %);
  - cada lectura guarda su posición real, para medir la exactitud del alineamiento.
- **k-mers:** comun/kmers.py los extrae codificados en 2 bits con una ventana deslizante en Θ(n).

Uso: `python -m datos.generar <largo_genoma> <num_lecturas> [semilla] [tasa_indel]`.

Prueba de punta a punta (`tests/test_comun_indice.py`): con un genoma de 30 000 bases, 100 lecturas con 2 % de sustituciones y 1 % de indels, y k = 12, el pipeline ubica las 100 lecturas a ≤ 3 bases de su posición real.